*BedRock Language & Compiler Architecture*

# BedRock: A Zero Runtime Compiler and Bare Metal Language

## ~12× Faster Compilation & ~3.8× Smaller Binaries Than GCC (RISC-V)

**Karim Mohamed Hanafy Abdelgawad**

Abdulrahman Ali El-Shennawy School | Systems Software (SOFT)

## ABSTRACT

I built BedRock, a systems programming language and multi target compiler, from scratch in Rust

I designed it for bare metal programming, where developers usually depend on a vendor SDK or a C runtime that hides how a chip boots and how source code becomes machine code. My compiler lowers BedRock source through a lexer, parser, type inference, verifier, optimizer, and a three address intermediate representation into native

**MIPS, RISC-V, and x86-64** machine code, with no C runtime.

I benchmarked five programs built for RISC-V. BedRock compiled about **12× faster** than GCC (-Os, freestanding) and produced binaries about **3.8× smaller**
(245 B vs 918 B on average).

I checked the RISC-V output byte by byte, in an emulator, and with a codegen test matrix, then ran it on a real **ESP32-C3** microcontroller with no operating system and no vendor SDK.

## BACKGROUND / INTRODUCTION

- Most embedded software sits on layers that hide the machine: a **vendor SDK** (such as ESP-IDF), a **C runtime** (startup code and libc), and a general purpose toolchain such as GCC.
- Those layers cost build time and binary size.
  They also make it hard to see how a chip really boots and which bytes actually run.
- I based BedRock on three axioms.
  **No Magic:** every operation maps to a fixed sequence of instructions, with no garbage collector or hidden allocation.
  **Byte Level Validation:** I can check the output in a hex editor, with a source map.
  **Direct Hardware Control:** peek, poke, inline asm, and interrupts.
- **The gap:** a small, readable compiler that goes from source to bare metal machine code with nothing hidden in between.

*BedRock at a glance: real code from the ESP32-C3 Morse decoder*

```
root GPIO_OUT_W1TC = 0x6000400C;
root LED_BIT = 1 << 8;

fn led_on() {
    poke(GPIO_OUT_W1TC, LED_BIT);
}

fn serial_send_char(c) {
    while ((peek(USB_EP1_CONF) & 2) == 0) { }
    poke(USB_EP1, c);
}
```

## RESEARCH QUESTION & ENGINEERING GOAL

**Research question**

**Can a from scratch compiled language generate working bare metal firmware for a real microcontroller, with no vendor SDK or operating system, while compiling faster and producing smaller binaries than GCC?**

**Engineering goals**

- I built a multi target compiler (MIPS, RISC-V, x86-64) in Rust with **no external crates**.
- I made every stage inspectable: IR dump (--emit-ir), source maps, and raw .bin output.
- I measured **compile time and binary size** against GCC -Os on the same RISC-V programs.
- I ran BedRock firmware on a real **ESP32-C3** through a direct boot path that bypasses the vendor bootloader.

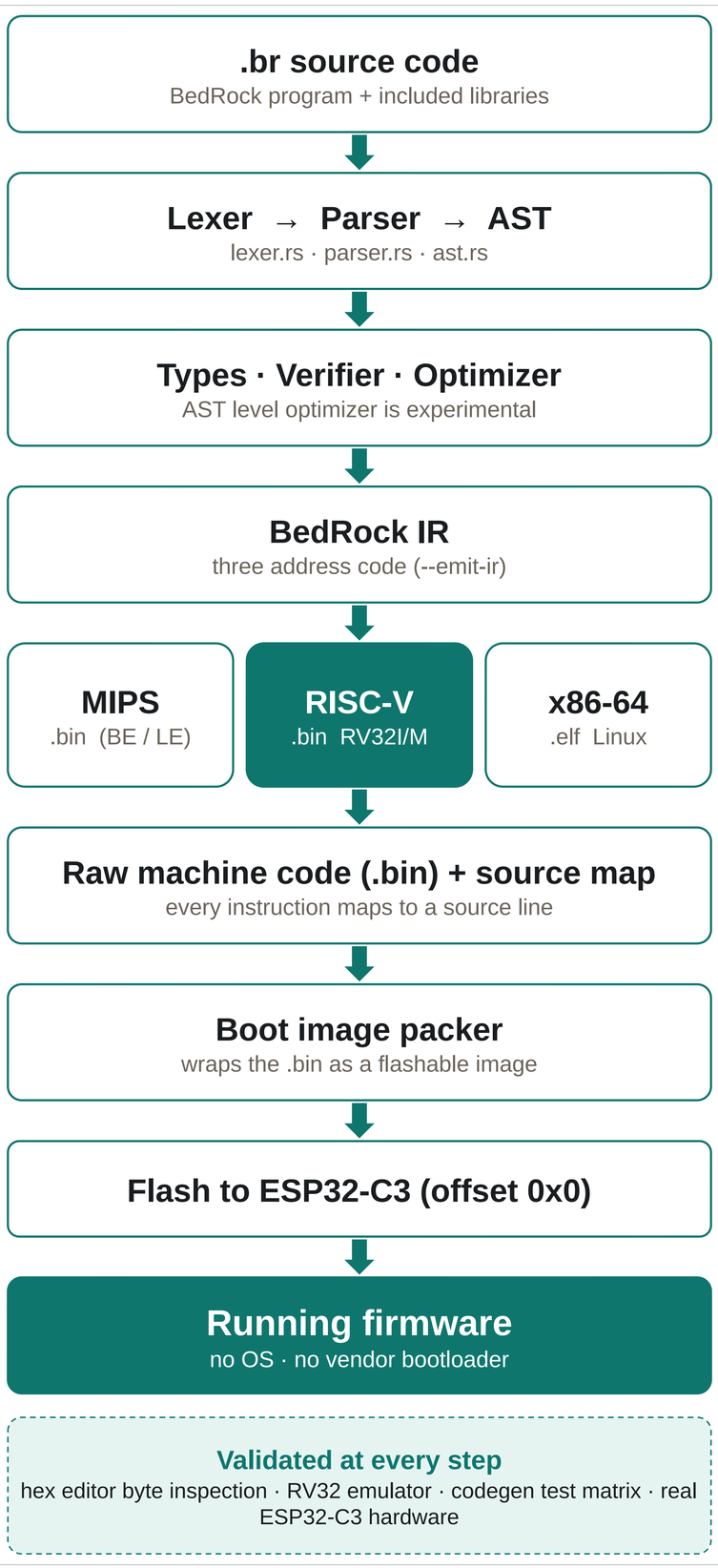
## METHODOLOGY

1. I designed the language and its three axioms, then wrote the compiler in Rust: lexer, parser, type inference, verifier, and optimizer.
2. I added **BedRock IR**, a three address intermediate form, and built backends for MIPS, RISC-V (RV32I/M), and x86-64 Linux ELF.
3. I wrote a codegen test matrix (immediates, arithmetic, registers, branches, calls, memory, GPIO, stress).
   I checked the output by reading binaries and running them in an RV32 emulator.
4. I fixed the bugs the tests exposed, such as live registers not preserved across nested calls.
5. I reverse engineered the ESP32-C3 boot image requirements from the esptool source and wrote a packer for **direct boot**.
6. I disarmed the RTC, timer group, and super watchdogs (SWD unlock key 0x8F1D312A) in BedRock code at the start of main, so direct boot runs with no watchdog resets.
7. I flashed real hardware and iterated using serial output and crash register dumps.
8. I benchmarked compile time and binary size against GCC (-Os, RV32IMC, freestanding) on five programs, with warm up runs before timing.

*RISC-V codegen test matrix*

| | | | |
|---|---|---|---|
| **CG-IMM** immediates | **CG-ARITH** arithmetic | **CG-REG** registers | **CG-BR** branches |
| **CG-CALL** calls | **CG-MEM** memory | **CG-GPIO** GPIO | **CG-STRESS** stress |

## SYSTEM DIAGRAM



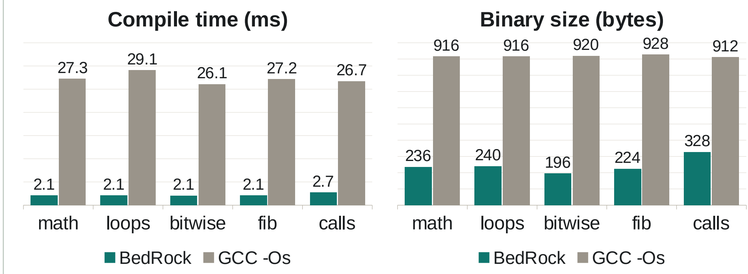

## RESULTS

**~12×** faster compile time than GCC (RISC-V)

**~3.8×** smaller binaries (245 B vs 918 B avg)

**~7,700** lines of Rust, zero external crates

**5 / 5** benchmarks built OK on both




*BedRock: brc3 --target riscv. GCC: riscv32 gcc -Os -march=rv32imc -ffreestanding -nostdlib. Mean of timed runs after warm up compiles. Size = BedRock raw .bin vs GCC linked ELF (a complete loadable image each). Lower is better. Compile time and size only, not runtime speed.*

- BedRock compiled in about **2 ms** on every benchmark.
  GCC needed **26 to 29 ms**.
- **On real hardware,** I ran a Morse code decoder and a gesture locked boot gate on the ESP32-C3.
  The decoder runs as a self contained image of about 5 kB, compiled at the default optimization level.
  Fixing real boot failures (missing load address, stack and data placed in read only memory) made them work.
- My RISC-V codegen baseline passes **7 of 7** tests (negative literals only partially).

## DISCUSSION

- **Why smaller and faster:** BedRock has no C runtime, no startup code, and no linker step, so there is less to compile and less to emit.
- **Fair comparison:** my earlier hosted x86 test showed 5.2× and 133×, but that GCC build included startup and linking.
  I use the like for like freestanding RISC-V result here.
- **Hard to debug:** a bad boot header, a missing load address, or a corrupt stack all look like the same reset loop.
  Reading RISC-V exception register dumps let me separate “never started” from “started, then crashed.”
- **Limits:** GCC supports a whole language and many targets.
  My optimizer is experimental and my ARM backend is only a stub.

## CONCLUSION & FUTURE WORK

Yes.
I showed that one developer can build a compiler to silicon pipeline end to end.
BedRock compiles about 12× faster and emits binaries about 3.8× smaller than GCC on RISC-V benchmarks.
Its firmware runs on a real ESP32-C3 with no operating system and no vendor bootloader.

**Future work**

- **BR3 Micro Kernel:** I plan to run several BedRock apps side by side, isolated with RISC-V Physical Memory Protection.
- I will measure **runtime** performance, not only compile time and size, and finish the ARM backend.
- Write a BedRock PC installer that uploads source for on device interpretation.

*Try it: BedRock is open source*

```
$ brc3 app.br --target riscv -o app.bin
$ brc3 app.br --target x86 -o app.elf
$ brc3 app.br --target ir
```

## AI USE DISCLOSURE

I used more than one AI tool, under my supervision, for debugging and generating parts of the compiler code writing the study guide and documentation, and drafting the text and layout of this poster.
I ran the benchmarks, tests, and hardware runs shown here on my own setup.

## ACKNOWLEDGMENTS

Thank you to Ms.Eman Askar for guidance throughout this project, and to the El-Shorouk Local Fair organizers and judges for their time.

## REFERENCES

Espressif Systems, ESP32-C3 Technical Reference Manual · Espressif Systems, esptool source and documentation · RISC-V International, The RISCV Instruction Set Manual GCC, RISC-V toolchain (-Os) BedRock documentation, bedrock.abrdns.com

## CONTACT / QR

**Left QR, source: github.com/RaspGrasp/BedRock-Language-Open-Source**
**Right QR, docs: bedrock.abrdns.com**